Fast Fourier Transform Implementation in MIPS Assembly

Thumu Abhinav

March 1, 2025

1 Introduction

The Fast Fourier Transform (FFT) is an efficient algorithm to compute the Discrete Fourier Transform (DFT). It reduces the computational complexity from $O(N^2)$ for the DFT to $O(N \log N)$ by recursively dividing the problem into smaller subproblems. This implementation focuses on computing the FFT for small values of N (specifically $N = 2$ and $N = 4$) using MIPS assembly language.

The key steps in the FFT algorithm are:

1. Splitting the input into even and odd indexed elements.
2. Recursively computing the FFT of the smaller subproblems.
3. Combining the results using the butterfly operation.

2 Implementation Details

The MIPS implementation is designed to handle small input sizes ($N = 2$ and $N = 4$) and demonstrates the core principles of the FFT algorithm. The implementation is divided into several key components:

2.1 Memory Organization

The input data and intermediate results are stored in memory as follows:

- Complex numbers are stored as pairs of 32-bit words (real part followed by imaginary part).
- The input array is allocated 32 bytes (8 words) to store up to 4 complex numbers.
- Twiddle factors (precomputed roots of unity) are stored in lookup tables:
  - `twiddle_n2`: $W_2^0 = 1$ and $W_2^1 = -1$.
  - `twiddle_n4`: $W_4^0 = 1$, $W_4^1 = -i$, $W_4^2 = -1$, and $W_4^3 = i$.

2.2 Key Implementation Steps

The implementation is divided into the following steps:

2.2.1 Input Handling

The program reads complex numbers from the user and stores them in the input array. Each complex number consists of a real and imaginary part, which are stored consecutively in memory.

```
input_loop:
    # Read real part
    li $v0, 4
    la $a0, prompt_real
    syscall
    li $v0, 5
    syscall
    move $t2, $v0           # Store real part in $t2

    # Read imaginary part
    li $v0, 4
    la $a0, prompt_imag
    syscall
    li $v0, 5
    syscall
    move $t3, $v0           # Store imaginary part in $t3

    # Store in array
    la $t4, input_array
    sll $t5, $t1, 3         # Calculate offset (i * 8)
    addu $t4, $t4, $t5      # Address of current element
    sw $t2, 0($t4)          # Store real part
    sw $t3, 4($t4)          # Store imaginary part
```

2.2.2 Base Case (N = 2)

For $N = 2$, the FFT is computed directly using the butterfly operation:

$$X[0] = x[0] + x[1], \quad X[1] = x[0] - x[1]$$

```
fft_n2:
    lw $t0, 0($s1)  # x[0].real
    lw $t1, 4($s1)  # x[0].imag
    lw $t2, 8($s1)  # x[1].real
    lw $t3, 12($s1) # x[1].imag

    add $t4, $t0, $t2   # X[0].real
    add $t5, $t1, $t3   # X[0].imag
    sub $t6, $t0, $t2   # X[1].real
```

```
    sub $t7, $t1, $t3    # X[1].imag

    sw $t4, 0($s1)       # Store X[0].real
    sw $t5, 4($s1)       # Store X[0].imag
    sw $t6, 8($s1)       # Store X[1].real
    sw $t7, 12($s1)      # Store X[1].imag
```

2.2.3 Recursive Case (N = 4)

For $N = 4$, the input is split into even and odd indexed elements, and the FFT is computed recursively. The results are combined using the butterfly operation with twiddle factors.

```
fft_n4:
    # Split into even and odd elements
    lw $t0, 0($s2)        # Even[0].real
    lw $t1, 4($s2)        # Even[0].imag
    lw $t2, 16($s2)       # Even[1].real
    lw $t3, 20($s2)       # Even[1].imag

    # Compute FFT of even elements
    li $a0, 2
    move $a1, $sp
    jal fft

    # Combine results with twiddle factors
    la $t8, twiddle_n4
    lw $t9, 0($t8)        # W4^0.real
    lw $t8, 4($t8)        # W4^0.imag
    mul $s4, $t2, $t9     # Real part
    mul $s5, $t3, $t8
    sub $s4, $s4, $s5     # Final real
    mul $s6, $t2, $t8
    mul $s7, $t3, $t9
    add $s5, $s6, $s7     # Final imag
```

2.2.4 Output Formatting

The results are printed in the format $a + bi$ or $a - bi$, depending on the sign of the imaginary part.

```
print_loop:
    # Load complex number
    lw $t2, 0($t4)       # Real part
    lw $t3, 4($t4)       # Imaginary part

    # Print real part
    li $v0, 1
    move $a0, $t2
```

```
    syscall

    # Print imaginary part
    bltz $t3, print_neg
    li $v0, 4
    la $a0, plus_sign
    syscall
    j print_imag
print_neg:
    li $v0, 4
    la $a0, minus_sign
    syscall
    sub $t3, $zero, $t3
print_imag:
    li $v0, 1
    move $a0, $t3
    syscall
    li $v0, 4
    la $a0, i_suffix
    syscall
```

3 Results

The program successfully computes the FFT for $N = 2$ and $N = 4$. The results are displayed in the format $a + bi$ or $a - bi$.

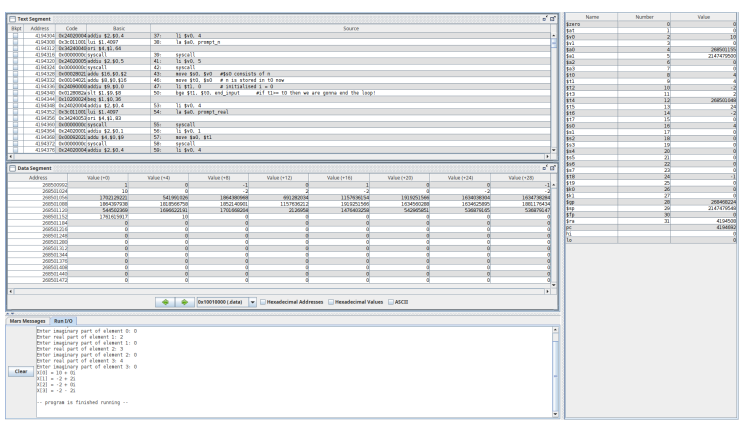

Figure 1: FFT Computation Results

Name	Number	Value
$zero	0	0
$at	1	0
$v0	2	0
$v1	3	0
$a0	4	0
$a1	5	0
$a2	6	0
$a3	7	0
$t0	8	0
$t1	9	0
$t2	10	0
$t3	11	0
$t4	12	0
$t5	13	0
$t6	14	0
$t7	15	0
$s0	16	0
$s1	17	0
$s2	18	0
$s3	19	0
$s4	20	0
$s5	21	0
$s6	22	0
$s7	23	0
$t8	24	0
$t9	25	0
$k0	26	0
$k1	27	0
$gp	28	268468224
$sp	29	2147479548
$fp	30	0
$ra	31	0
pc		4194304
hi		0
lo		0

Figure 2: Initial Register States Before FFT Computation

Name	Number	Value
$zero	0	0
$at	1	0
$v0	2	10
$v1	3	0
$a0	4	268501155
$a1	5	2147479500
$a2	6	0
$a3	7	0
$t0	8	4
$t1	9	4
$t2	10	-2
$t3	11	2
$t4	12	268501048
$t5	13	24
$t6	14	-2
$t7	15	0
$s0	16	4
$s1	17	0
$s2	18	0
$s3	19	0
$s4	20	0
$s5	21	0
$s6	22	0
$s7	23	0
$t8	24	-1
$t9	25	0
$k0	26	0
$k1	27	0
$gp	28	268468224
$sp	29	2147479548
$fp	30	0
$ra	31	4194508
pc		4194692
hi		0
lo		0

Figure 3: Final Register States After FFT Computation

4 Conclusion

This implementation demonstrates the core principles of the FFT algorithm in MIPS assembly language. It handles small input sizes efficiently and showcases:

- Recursive problem decomposition.
- Complex number arithmetic.
- Memory management and stack usage.
- System call integration for input/output.